

第四章：语法分析

递归下降法

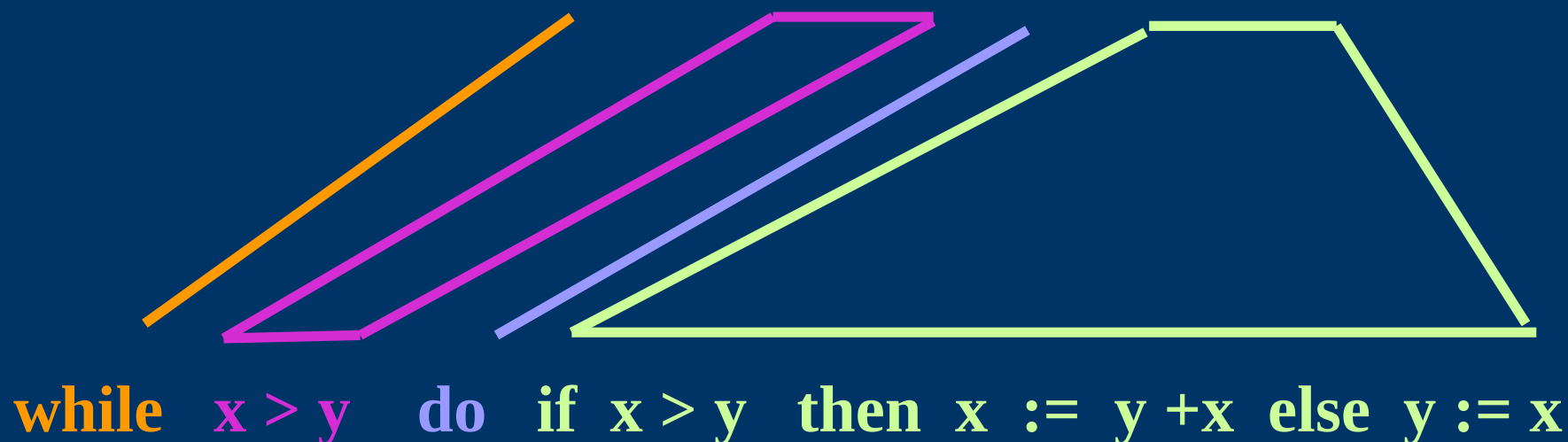
1.1 递归下降法的基本原理

- ◆ 递归下降分析法是一种自顶向下语法分析方法
- ◆ 采用分而治之的思想：若想识别一个语法结构，可以按照产生式规则将其分成几个部分来识别。

其中终极符直接识别，非终极符则调用相应子程序识别

- ◆ 例如：有产生式

$\langle \text{While_Stm} \rangle \rightarrow \text{while } \langle \text{Exp} \rangle \text{ do } \langle \text{Stm} \rangle$



1.1 递归下降法的基本原理

- ◆ While_Stm的递归子程序：作用是识别、分析由While_Stm推导出的字符串。

$\langle \text{While_Stm} \rangle \rightarrow \text{while } \langle \text{Exp} \rangle \text{ do } \langle \text{Stm} \rangle$

```
void While_Stm()
```

```
{  Match(while);    //终极符匹配
```

```
   Exp();           //非终极符调用子程序识别对应部分
```

```
   Match(do);       //终极符匹配
```

```
   Stm();           //非终极符调用子程序识别对应部分
```

```
}    /*此为示意程序，非真实程序，真实程序还要增加一些  
     细节处理*/
```

1.1 递归下降法的基本原理

- ◆ 当非终极符有多个候选式时，根据候选式的Predict集选择候选式进行识别。

$\langle \text{While_Stm} \rangle \rightarrow \text{while } \langle \text{Exp} \rangle \text{ do } \langle \text{Stm} \rangle$
 $\quad \quad \quad | \text{do } \langle \text{Stm} \rangle \text{ while } \langle \text{Exp} \rangle$

```
void While_Stm()
```

```
{ if (token==while) { Match(while); Exp(); Match(do); Stm();}  
  else if (token==do) { Match(do); Stm();Match(while); Exp();}  
  else error();  
}
```

/*在调用函数While_Stm之前， token中已读入当前输入符*/

1.2 对文法的要求

- ◆ 为了保证推导的唯一性，递归下降分析法对文法的要求与LL(1)文法相同。
- ◆ 即对于文法G中任一非终极符A，其任意两个产生式 $A \rightarrow \alpha$ 和 $A \rightarrow \beta$ ，都要满足下面条件：

$$\text{Predict}(A \rightarrow \alpha) \cap \text{Predict}(A \rightarrow \beta) = \emptyset$$

或者，等价表示：

$$A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_i \mid \dots \mid \alpha_j \mid \dots \mid \alpha_n$$

$$\text{First}(\alpha_i) \cap \text{First}(\alpha_j) = \emptyset,$$

$$\text{First}(\alpha_i) \text{ 不含 } \varepsilon, i=1 \sim n$$

$$\text{First}(\alpha_i) \cap (\text{Follow}(A) \cup \text{First}(\alpha_j) - \{\varepsilon\}) = \emptyset,$$

$$\text{First}(\alpha_i) \text{ 不含 } \varepsilon, \text{First}(\alpha_j) \text{ 含 } \varepsilon$$

2. 递归下降语法分析程序的构造

- ◆ 递归下降语法分析程序构成：
- ◆ 一个全局变量token：用于记录输入流的当前符号
- ◆ 每个 V_N 一个函数：有多个候选式，用predict集选择识别分支
- ◆ 一个主函数：用于调用开始符函数
- ◆ 两个辅助函数
 1. ReadToken()：把输入流的当前符号读入token中
 2. Match('a'): if (token == 'a') ReadToken () ;
else error();

/*Match函数实现终极符匹配，如成功则读入下一个字符，否则失败*/

2. 递归下降语法分析程序的构造

当产生式形如: $A \rightarrow \beta_1 | \beta_2 | \dots | \beta_n$, 则按下面的方法编写子程序A:

```
void A( )  
{ if token  $\in$  Predict( $A \rightarrow \beta_1$ ) then  $\theta(\beta_1)$  else  
  if token  $\in$  Predict( $A \rightarrow \beta_2$ ) then  $\theta(\beta_2)$  else  
    .....  
    if token  $\in$  Predict( $A \rightarrow \beta_n$ ) then  $\theta(\beta_n)$  else  
      error( )  
}
```

其中对 $\beta_i = X_1 X_2 \dots X_n$, $\theta(\beta_i) = \theta'(X_1); \theta'(X_2); \dots; \theta'(X_n)$;

如果 $X \in V_N$, $\theta'(X) = X()$; //调用非终极符X子程序

如果 $X \in V_T$, $\theta'(X) = \text{Match}(X)$; //终极符X匹配;

如果 $X = \varepsilon$, $\theta'(\varepsilon) = \text{skip}$ (空语句).

2. 递归下降语法分析程序的构造

- ◆ 主程序:

```
void main(){
    ReadToken();           //读输入流的第一个符号
    S ();                  //从开始符开始识别
    Match( '#' );          //开始符识别结束, 判断输入流是否
                           //到末尾则成功 (#是指定的输入流结
                           //束符, 也可指定其他结束符), 否则报错
}
```


例：假设有文法

$$Z \rightarrow a B a$$
$$B \rightarrow b B \mid c$$

$\text{Predict}(Z \rightarrow aBa) = \{a\},$

$\text{Predict}(B \rightarrow bB) = \{b\}, \text{Predict}\{B \rightarrow c\} = \{c\}$

则相应的递归子程序可如下：

void Z()

{

if (token=='a') { Match('a');

B();

Match('a') ;}

else error(1);

void B ()

{

if (token == ' b') {Match('b');

B();}

else if (token == 'c') Match('c');

else error(2);

}

}

语法分析主程序： **void main(){ReadToken(); Z();Match('#');}**

例:	$E \rightarrow T E'$	$\text{Predict}(E \rightarrow T E') = \text{first}(T E') = \{ i, (\}$
	$E' \rightarrow + T E' \mid \varepsilon$	$\text{Predict}(E' \rightarrow + T E') = \text{first}(+ T E') = \{ + \}$
	$T \rightarrow F T'$	$\text{Predict}(E' \rightarrow \varepsilon) = \text{follow}(E') = \{), \# \}$
	$T' \rightarrow * F T' \mid \varepsilon$	$\text{Predict}(T \rightarrow F T') = \text{first}(F T') = \{ i, (\}$
	$F \rightarrow i \mid (E)$	$\text{Predict}(T' \rightarrow * F T') = \text{first}(* F T') = \{ * \}$
		$\text{Predict}(T' \rightarrow \varepsilon) = \text{follow}(T') = \{ +,), \# \}$
		$\text{Predict}(F \rightarrow \text{id}) = \text{first}(\text{id}) = \{ i \}$
		$\text{Predict}(F \rightarrow (E)) = \text{first}((E)) = \{ (\}$

语法分析主程序: `int main(){ ReadToken(); E() ; Match( '#' );}`

```
void E( )
{
    if (token==' i' || token==' (') {T(); E'() ; }
    else error( 1 );
}
```

例:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \varepsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \varepsilon$

$F \rightarrow i \mid (E)$

$\text{Predict}(E \rightarrow T E') = \text{first}(T E') = \{ i, (\}$

$\text{Predict}(E' \rightarrow + T E') = \text{first}(+ T E') = \{ + \}$

$\text{Predict}(E' \rightarrow \varepsilon) = \text{follow}(E') = \{), \# \}$

$\text{Predict}(T \rightarrow F T') = \text{first}(F T') = \{ i, (\}$

$\text{Predict}(T' \rightarrow * F T') = \text{first}(* F T') = \{ * \}$

$\text{Predict}(T' \rightarrow \varepsilon) = \text{follow}(T') = \{ +,), \# \}$

$\text{Predict}(F \rightarrow i) = \text{first}(i) = \{ i \}$

$\text{Predict}(F \rightarrow (E)) = \text{first}((E)) = \{ (\}$

void E' ()

{

if (token=='+') { Match('+');

T();

E' ();}

else if (token==')' || token==' #') skip; //E' → ε

else error(2);

}

例:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \varepsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \varepsilon$

$F \rightarrow i \mid (E)$

$\text{Predict}(E \rightarrow T E') = \text{first}(T E') = \{ i, (\}$

$\text{Predict}(E' \rightarrow + T E') = \text{first}(+ T E') = \{ + \}$

$\text{Predict}(E' \rightarrow \varepsilon) = \text{follow}(E') = \{), \# \}$

$\text{Predict}(T \rightarrow F T') = \text{first}(F T') = \{ i, (\}$

$\text{Predict}(T' \rightarrow * F T') = \text{first}(* F T') = \{ * \}$

$\text{Predict}(T' \rightarrow \varepsilon) = \text{follow}(T') = \{ +,), \# \}$

$\text{Predict}(F \rightarrow i) = \text{first}(i) = \{ i \}$

$\text{Predict}(F \rightarrow (E)) = \text{first}((E)) = \{ (\}$

void T()

{

if (token==' i' || token==' (') { F(); T '()); }

else error(3);

}

例:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \varepsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \varepsilon$

$F \rightarrow i \mid (E)$

$\text{Predict}(E \rightarrow T E') = \text{first}(T E') = \{ i, (\}$

$\text{Predict}(E' \rightarrow + T E') = \text{first}(+ T E') = \{ + \}$

$\text{Predict}(E' \rightarrow \varepsilon) = \text{follow}(E') = \{), \# \}$

$\text{Predict}(T \rightarrow F T') = \text{first}(F T') = \{ i, (\}$

$\text{Predict}(T' \rightarrow * F T') = \text{first}(* F T') = \{ * \}$

$\text{Predict}(T' \rightarrow \varepsilon) = \text{follow}(T') = \{ +,), \# \}$

$\text{Predict}(F \rightarrow i) = \text{first}(i) = \{ i \}$

$\text{Predict}(F \rightarrow (E)) = \text{first}((E)) = \{ (\}$

void T' ()

{

if (token == '*') { Match('*');

F();

T' ();}

else if (token == '+' || token == ') ' || token == '#') skip; // T' → ε

else error(4);

}

例:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \varepsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \varepsilon$

$F \rightarrow i \mid (E)$

$\text{Predict}(E \rightarrow T E') = \text{first}(T E') = \{ i, (\}$

$\text{Predict}(E' \rightarrow + T E') = \text{first}(+ T E') = \{ + \}$

$\text{Predict}(E' \rightarrow \varepsilon) = \text{follow}(E') = \{), \# \}$

$\text{Predict}(T \rightarrow F T') = \text{first}(F T') = \{ i, (\}$

$\text{Predict}(T' \rightarrow * F T') = \text{first}(* F T') = \{ * \}$

$\text{Predict}(T' \rightarrow \varepsilon) = \text{follow}(T') = \{ +,), \# \}$

$\text{Predict}(F \rightarrow i) = \text{first}(i) = \{ i \}$

$\text{Predict}(F \rightarrow (E)) = \text{first}((E)) = \{ (\}$

void F ()

{

if (token == ' i ') Match(' i ');

else if (token == ' (') { Match(' (');

E();

Match(') '); }

else error(5);

}

E→TE'

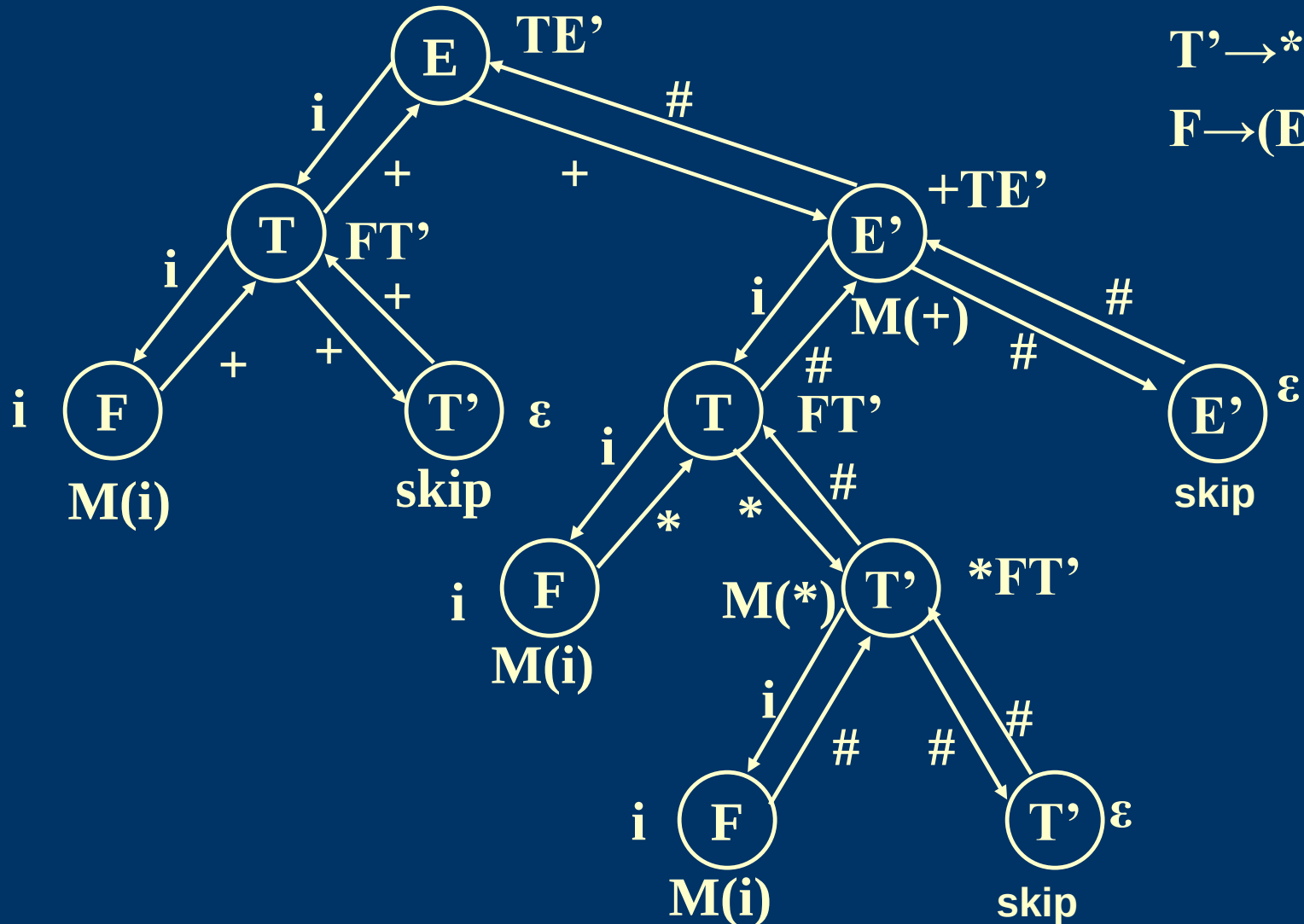
语法分析主程序: `int main(){ReadToken(); E() ; Match('#');}`

$$\mathbf{E}' \rightarrow +\mathbf{T}\mathbf{E}' \quad | \quad \varepsilon$$

T→FT'

$$T' \rightarrow *FT' \mid \varepsilon$$

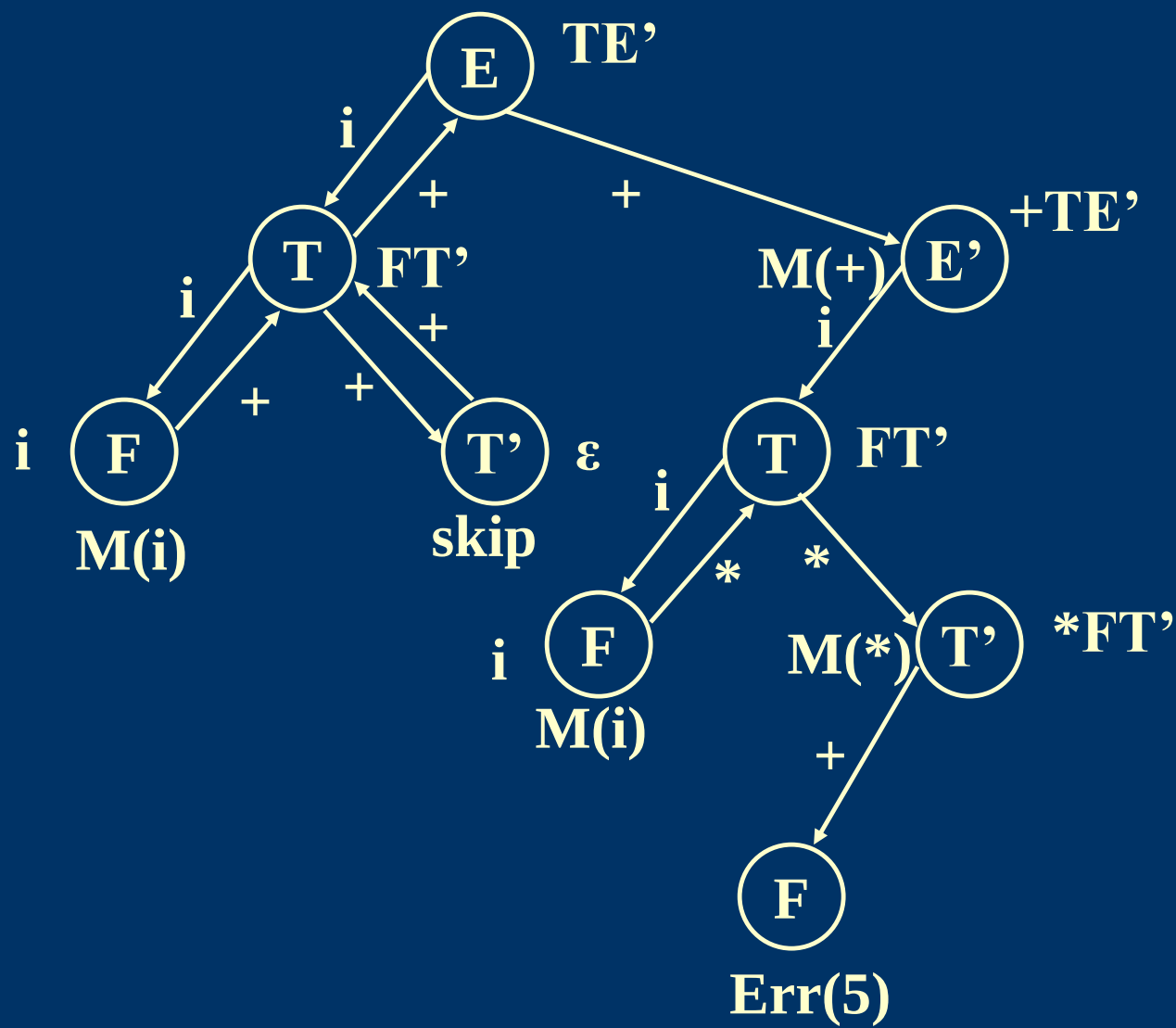
F→(E) | i



例： $i+i*+i \#$ 递归下降分析过程

语法分析主程序： `void main(){ReadToken(); E() ; Match('#');}`

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid i$



2. 递归下降语法分析程序的构造

- ◆ 递归下降分析法名字的由来：
- ◆ 终极符产生匹配命令，而非终极符则产生调用命令.因为文法递归相应子程序也递归，所以称这种方法为递归子程序方法或递归下降法。
- ◆ 分析能力：与LL(1)等同，对文法限制也相同
- ◆ 优点：构造简单
- ◆ 缺点：
 1. 程序比较长
 2. 频繁的函数调用影响效率
 3. 文法改变，语法分析程序要随之变动，而LL(1)只需修改LL(1)分析表

课后练习

已知文法G[S]:

$$S \rightarrow AB \mid bC$$
$$A \rightarrow \varepsilon \mid b$$
$$B \rightarrow \varepsilon \mid aD$$
$$C \rightarrow AD \mid b$$
$$D \rightarrow aS \mid c$$

- 1、计算每个非终极符的First集、Follow集.
- 2、计算每个产生式的Predict集合.
- 3、判断该文法是否为递归下降文法?

$S \rightarrow AB \mid bC$

$A \rightarrow \varepsilon \mid b$

$B \rightarrow \varepsilon \mid aD$

$C \rightarrow AD \mid b$

$D \rightarrow aS \mid c$

文法 符号	First集		
	step ₁	step ₂	step ₃
S	{ }	{b }	{b, a, ε }
A	{ε}	{b, ε }	{b, ε }
B	{ε}	{a, ε }	{a, ε }
C	{ }	{b}	{b,a,c}
D	{ }	{a, c }	{a, c }

$S \rightarrow AB \mid bC$

$A \rightarrow \varepsilon \mid b$

$B \rightarrow \varepsilon \mid aD$

$C \rightarrow AD \mid b$

$D \rightarrow aS \mid c$

文法符号	First集
S	{ a, b, ε }
A	{ b, ε }
B	{ a, ε }
C	{ a, b, c }
D	{ a, c }

文法符号	Follow集					
	step1	$S \rightarrow AB$	$S \rightarrow bC$	$B \rightarrow aD$	$C \rightarrow AD$	$D \rightarrow aS$
S	{#}	{#}	{#}	{#}	{#}	
A	{ }	{a,# }	{a,# }	{a,# }	{a,#,c }	
B	{ }	{#}	{#}	{#}	{#}	
C	{ }	{ }	{#}	{#}	{#}	
D	{ }	{ }	{ }	{#}	{#}	

$\text{Predict}(S \rightarrow AB) = \{ a, b, \# \}$

$\text{Predict}(S \rightarrow bC) = \{ b \}$

$\text{Predict}(A \rightarrow \varepsilon) = \{ a, c, \# \}$

$\text{Predict}(A \rightarrow b) = \{ b \}$

$\text{Predict}(B \rightarrow \varepsilon) = \{ \# \}$

$\text{Predict}(B \rightarrow aD) = \{ a \}$

$\text{Predict}(C \rightarrow AD) = \{ a, b, c \}$

$\text{Predict}(C \rightarrow b) = \{ b \}$

由于: **$\text{Predict}(S \rightarrow AB) \cap \text{Predict}(S \rightarrow bC) = \{ b \}$**

$\text{Predict}(C \rightarrow AD) \cap \text{Predict}(C \rightarrow b) = \{ b \}$

所以该文法不是递归下降文法.

文法 符号	First集	Follow集
S	{ a, b, ε }	{ # }
A	{ b, ε }	{ a, c, # }
B	{ a, ε }	{ # }
C	{ a, b, c }	{ # }
D	{ a, c }	{ # }